

What is Database Sharding?

#11 System Design - Database Sharding



ASHISH PRATAP SINGH

MAY 12, 2024



170



10



14

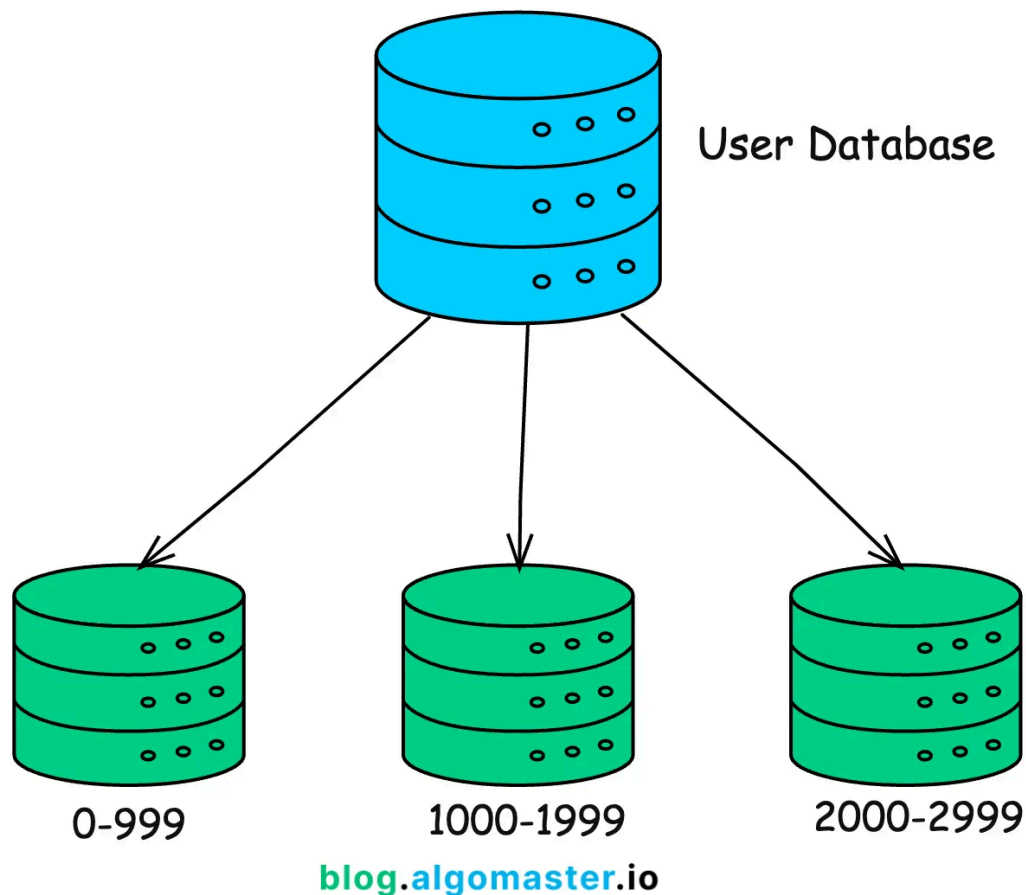
Share

Imagine a social media site like Instagram, which has over **1 billion** active users.

Think about what would happen if it tried to keep all the **user profile** data on a single server.

Due to limited scalability of a single machine, it would quickly run out of storage space and slow down leading to performance issues.

But what if we divided the user base into smaller groups based on a key like **userId** and stored each group on separate servers?



Example:

- Group 1: Users with IDs 0-999
- Group 2: Users with IDs 1000-1999
- Group 3: Users with IDs 2000-2999

Now, a single server only deals with subset of data.

Distributing data in this way makes it easier to scale and manage more users.

This is the idea behind **Database Sharding**.

In this article, we will explore what database sharding is, it's benefits, how it work challenges that come with it and the best practices for implementing it.

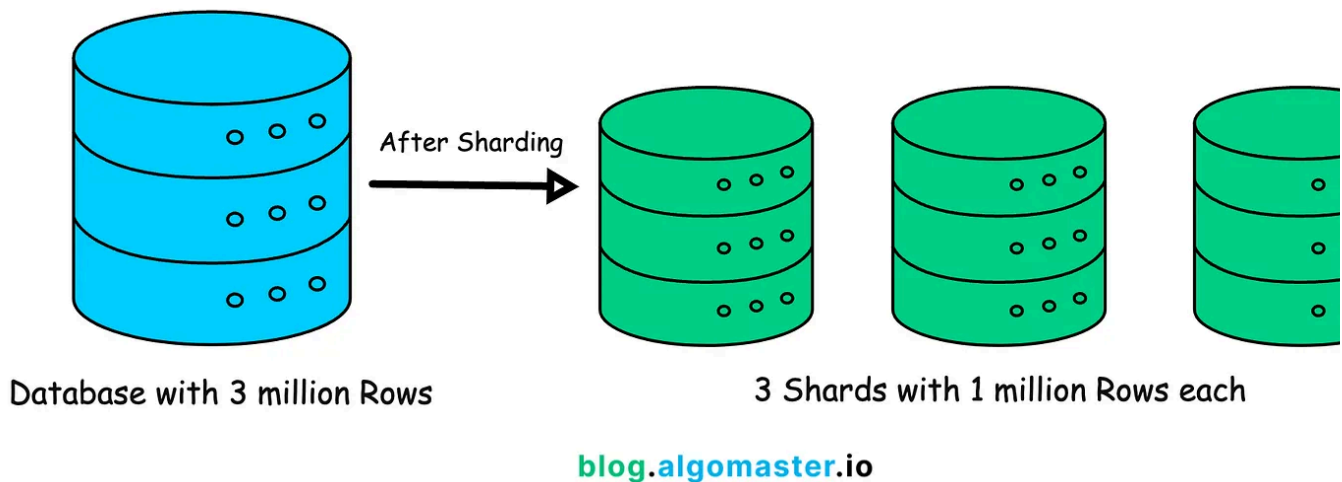
If you're enjoying this newsletter and want to get even more value, consider becon a [paid subscriber](#).

As a paid subscriber, you'll unlock all **premium** articles and gain full access to all [premium courses](#) on [algomaster.io](#).

1. What is Database Sharding?

Database sharding is a horizontal scaling technique used to split a large database smaller, independent pieces called **shards**.

These shards are then distributed across multiple servers or nodes, each responsible for handling a specific subset of the data.



Sharding is widely used by large-scale applications and services that handle massive amounts of data, such as:

- **Social Networks:** Instagram uses sharding to manage billions of user profiles interactions.
- **E-commerce Platforms:** Amazon employs sharding to handle massive product catalogs and customer data.
- **Search Engines:** Google relies on sharding to index and retrieve billions of web pages.
- **Gaming:** Online gaming platforms use sharding to handle millions of players vast amounts of game data.

2. Benefits of Sharding

1. **Improved Performance:** By distributing the data across multiple nodes, sharding can significantly reduce the load on any single server, resulting in faster query execution and improved overall system performance.
2. **Scalability:** Sharding allows databases to grow horizontally. As data volume increases, new shards can be added to spread the load evenly across the cluster.
3. **High Availability:** With data spread across multiple shards, the failure of a single shard doesn't bring down the entire system. Other shards can continue serving requests, ensuring high availability.
4. **Geographical Distribution:** Sharding allows you to strategically place shards closer to your users, reducing latency and improving the user experience.
5. **Reduced Cost:** Instead of scaling vertically by investing in more powerful and expensive hardware, sharding allows for cost-effective scaling by utilizing commodity hardware.

3. How Does Sharding Work?

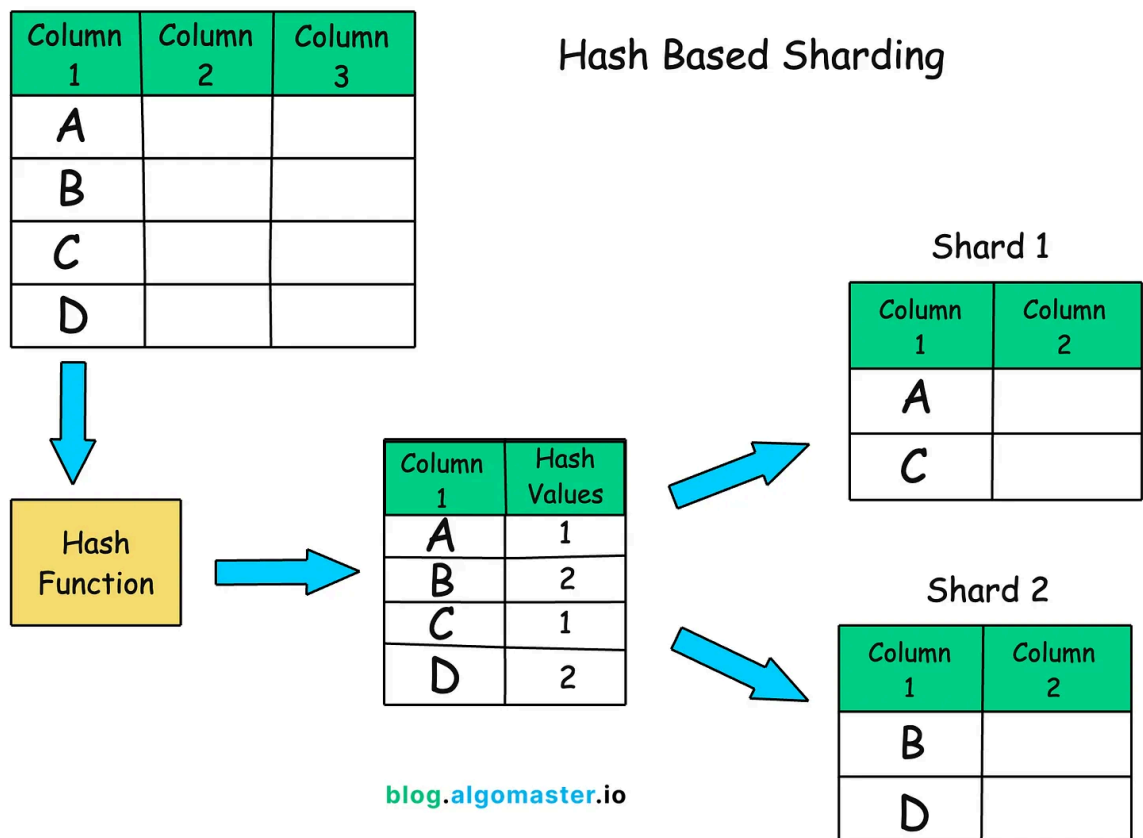
The sharding process involves several key components including:

1. **Sharding Key:** The shard key is a unique identifier used to determine which shard a particular piece of data belongs to. It can be a single column or a combination of columns.
2. **Data Partitioning:** Partitioning involves splitting the data into shards based on the shard key. Each shard represents a portion of the total data set. Common strategies to partition database are **range-based**, **hash-based**, and **directory-based sharding**.
3. **Shard Mapping:** Creating a mapping of shard keys to shard locations.

4. **Shard Management:** The shard manager oversees the distribution of data across shards, ensuring data consistency and integrity.
5. **Query Routing:** The query router intercepts incoming queries and directs them to the appropriate shard(s) based on the shard key.

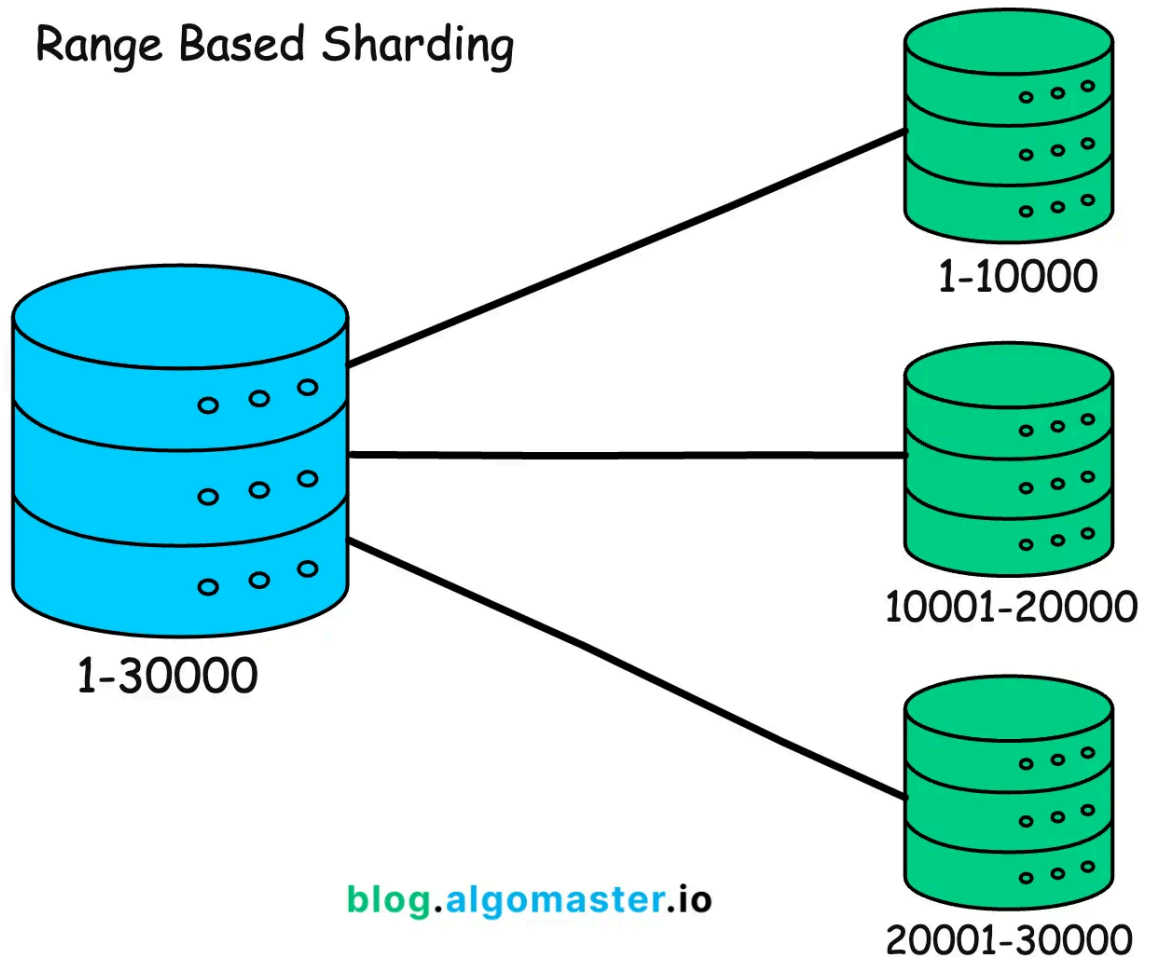
4. Sharding Strategies

- **Hash-Based Sharding:** Data is distributed using a hash function, which maps to a specific shard.
 - **Example:** $\text{Hash}(\text{user_id}) \% 2$ determines the shard number for a user, distributing users evenly across 2 shards.



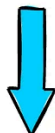
- **Range-Based Sharding:** Data is distributed based on a range of values, such as dates or numbers.
 - **Example:** Shard 1 contains records with IDs from 1 to 10000, Shard 2 contains records with IDs from 10001 to 20000, and so on.

Range Based Sharding



- **Geo-Based Sharding:** Data is distributed based on geographic location.
 - **Example:** Shard 1 serves users in North America, Shard 2 serves users in Europe, Shard 3 serves users in Asia.
- **Directory-Based Sharding:** Maintains a lookup table that directly maps specific keys to specific shards.

Column 1	Column 2	Column 3
A		
B		
C		
D		



Column 1	Shard ID
A	S1
B	S2
C	S3
D	S4



Directory Based Sharding

blog.algomaster.io

S1		S3	
Column 1	Column 2	Column 1	Column 2
A		C	
S2		S4	
Column 1	Column 2	Column 1	Column 2
B		D	

5. Challenges with Sharding

1. **Complexity:** Sharding introduces additional complexity, requiring careful planning and management.
2. **Data Consistency:** Maintaining data consistency across shards can be challenging especially when data needs to be joined or aggregated from multiple shards.
3. **Cross-shard Joins:** Performing joins across multiple shards can be complex and computationally expensive.
4. **Data Rebalancing:** As data volumes grow, shards may become unevenly distributed, requiring rebalancing to maintain optimal performance.

6. Best Practices for Sharding

- **Choose the Right Sharding Key:** Select a sharding key that ensures an even distribution of data across shards and aligns with the application's access patterns.
- **Use Consistent Hashing:** Implement a consistent hashing algorithm to minimize data movement when adding or removing shards.
- **Monitor and Rebalance Shards:** Regularly monitor shard performance and data distribution. Rebalance shards as needed to ensure optimal performance and data distribution.
- **Handle Cross-Shard Queries Efficiently:** Optimize queries that require data from multiple shards by using techniques like query federation or data denormalization.
- **Plan for Scalability:** Design your sharding strategy with future growth in mind. Consider how the system will scale as data volume and traffic increase.

7. Conclusion

To summarize, database sharding is a powerful technique for scaling databases horizontally and handling large amounts of data, but it does come with additional complexity and requires thoughtful planning and understanding of application's data access patterns.

Before implementing sharding, think about whether the benefits outweigh the costs if there is a simpler solution.

Thank you for reading!

If you found it valuable, hit a like ❤️ and consider subscribing for more such content every week.

If you have any questions or suggestions, leave a comment.

This post is public so feel free to share it.

P.S. If you're enjoying this newsletter and want to get even more value, consider becoming a [paid subscriber](#).

As a paid subscriber, you'll unlock all **premium** articles and gain full access to all [premium courses](#) on [algomaster.io](#).

There are [group discounts](#), [gift options](#), and [referral bonuses](#) available.

Checkout my [Youtube channel](#) for more in-depth content.

Follow me on [LinkedIn](#), [X](#) and [Medium](#) to stay updated.

Checkout my [GitHub repositories](#) for free interview preparation resources.

I hope you have a lovely day!

See you soon,

Ashish



170 Likes · 14 Restacks

← Previous

Next

Discussion about this post

Comments Restacks



Write a comment...



Debojit May 12, 2024

♥ Liked by Ashish Pratap Singh

Nice article Ashish!

♥ LIKE (2) 💬 REPLY

1 reply by Ashish Pratap Singh



Deepak Kumar May 12, 2024

♥ Liked by Ashish Pratap Singh

Well written Ashish!

Thank you for this contribution

♥ LIKE (2) 💬 REPLY

1 reply by Ashish Pratap Singh

8 more comments...